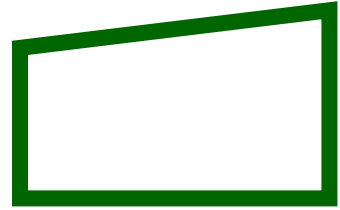


Module 2: Introduction to Automata Theory & Formal Languages

Professor: Dr. Arnold B. Galve
abgalve@kld.edu.ph

Learning Outcomes



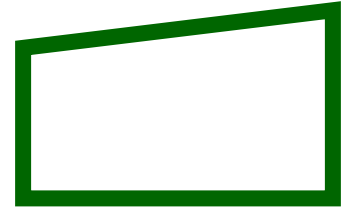
Explain the importance of automata and formal languages in computer science.

Define automata, formal languages, alphabets, strings, and grammars.

Describe the relationship among automata, languages, and computation.

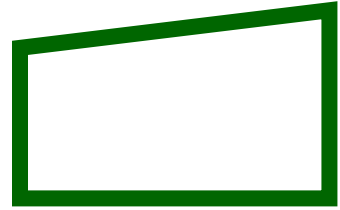
Identify practical applications of automata theory.

Primer



*How does a computer know whether your password,
email address, or program syntax is correct?*

Familiar Examples



ATM Pin Validation

Login Authentication

Email format checking

Search engines

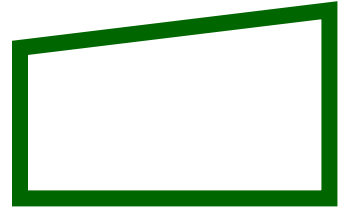
Spell Checkers

Programming language syntax

Chatbots

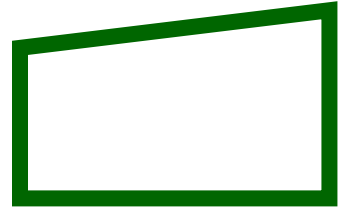
Voice assistants

What is Automata?



- Automata is defined as the mathematical **study of abstract computing machines** and the problems they can solve.
- It answers the following questions:
 - What can computers compute?
 - How do computers recognize patterns?
 - What are the limits of computations?
- Automata deals with **abstract models**, not physical machines.

Historical Background



- An English mathematician known today as the Father of Modern Computer Science and Artificial Intelligence.
- His foundational work laid the mathematical blueprint for modern programmable computers long before the technology existed.
- **Turing Machine:** He introduced a theoretical computing model that uses a simulated tape and specific algorithms to process information.



Alan Turing
(1912-1954)

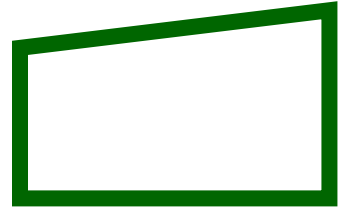
Historical Background



Alonzo Church
(1903-1995)

- He is a pioneering American mathematician and logician. He laid the foundational building blocks for theoretical computer science and mathematical logic.
- **Church-Turing Thesis:** This hypothesis states that our intuitive notion of an algorithm matches what can be computed by a Turing machine which defines the absolute limits of what computers can solve.

Historical Background

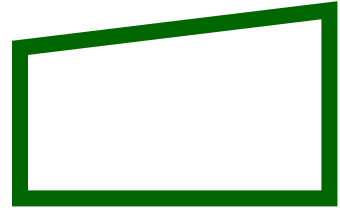


- American mathematician and logician who co-founded recursion theory, laying the foundational groundwork for theoretical computer science.
- Established the core boundaries of what functions can be calculated by machines.
- He is also widely celebrated for inventing **regular expressions** (regex), an indispensable tool used daily by software developers worldwide for pattern matching and text processing.



Stephen Kleene
(1909-1994)

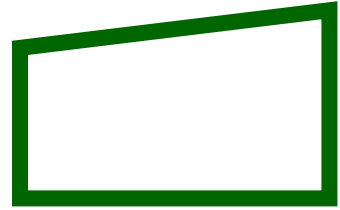
Historical Background



John Von Neumann
(1903-1957)

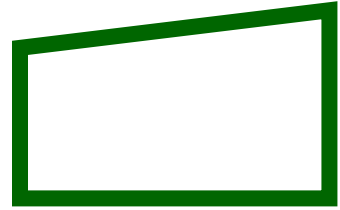
- Hungarian-American polymath, mathematician, physicist, and computer scientist who is widely considered **one of the greatest scientific minds in human history**.
- **Core Components:** He formalized the digital computer layout consisting of a Central Processing Unit (CPU), control unit, memory, and Input/Output (I/O) mechanics.
- **Automata Theory:** He pioneered the field of cellular automata and designed the theoretical universal constructor, which laid the early mathematical groundwork for self-replicating systems and robotics.

What is a Formal Language



- A **formal language** is a set of strings created according to precise mathematical rules.
- There are two types of languages:
 - Informal Languages (Semantic languages)
 - Formal Languages (Syntactic languages)

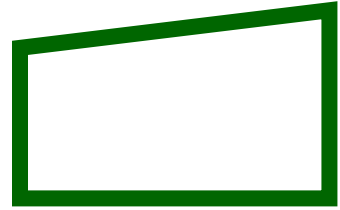
Formal Languages



- **Formal Language**

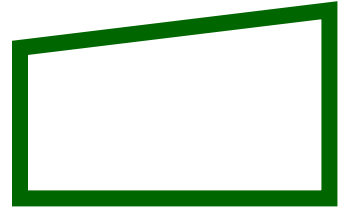
- Java
 - Python syntax
 - SQL
 - Mathematical notations
 - Regular expressions
-
- We must make and consider all the possible strings/subsets according to the predefined rules.
-
- It can be finite or infinite.

Informal Languages



- Does not strictly follow the grammar rules and we divert somehow from the strict grammar rules.
- **Examples:**
 - English
 - Filipino
 - Japanese
 - Arabic

Formal Languages



- **Example:**

$A = \{1, 2, 3\}$

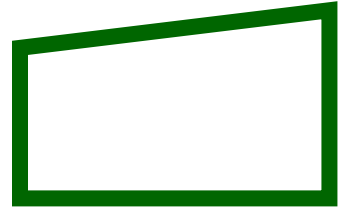
Rule1: Make a set of all subsets of A

[$\{\}$, $\{1\}$, $\{2\}$, $\{3\}$, $\{1,2\}$, $\{1,3\}$, $\{2,3\}$, $\{1,2,3\}$]

Rule2: Make a set of all subsets containing 2 members

[$\{1,2\}$, $\{1,3\}$, $\{2,3\}$]

English as a Formal Language



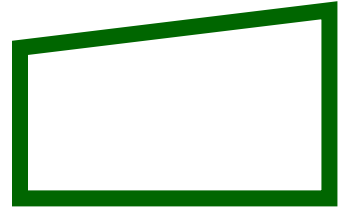
- We just follow the syntactic rules and do not care about the meanings of the sentences.
- Example: **S + V + O (Syntactic Rule)**

I eat apples (S + V + O)

Table eat apples (S + V + O)

Tree walk on road (S + V + O)

Properties of Formal Languages



Alphabets

Tokenizing (Factorization)

Strings

Words

Length of a String

Reverse of a String

Alphabets

Definition:

- A finite non-empty set of symbols (letters), is called an alphabet. It is denoted by Σ (Greek letter sigma), also called the set of units elements.
- According to the nature of problem (formal language) any symbol can be part of Σ

Alphabets

- Example:

$$\Sigma_1 = \{a, b\}$$

$$\Sigma_2 = \{0, 1\}$$

$$\Sigma_3 = \{i, j, 3, \#\}$$

Strings based upon Σ_1 may be as follows:

"aaa", "bbab" , "bbbb" , etc.

Tokenizing

⌘ Process of Breaking a word/string according to the given alphabet.

⌘ Example

☐ $\Sigma = \{a, b\}$ Given String; $S = \text{"aababa"}$

☐ (a) (a) (b) (a) (b) (a)

Strings

⌘ Definition:

Concatenation of finite symbols from the alphabet is called a string.

⌘ Example:

If $\Sigma = \{a, b\}$ then strings can be:

a, abab, aaabb, ababababababababab,...

NOTE:

Empty String or Null String

⌘ Sometimes a string with no symbol at all is used, denoted by (Small Greek letter Lambda) λ or (Capital Greek letter Lambda) Λ , is called an empty string or null string.

The capital lambda will be used to denote the empty string in this text.

Words

⌘ Definition:

Words are the strings belonging to some language/set.

Example:

If $\Sigma = \{x\}$ then a language/set L can be defined as: $L = \{x^n : n = 1, 2, 3, \dots\}$ or $L = \{x, xx, xxx, \dots\}$ Here $x, xx, \dots$ are the words of L .

NOTE:

⌘ All words are strings, but not all strings are words.

Length of Strings

⌘ Definition:

The length of string s , denoted by $|s|$, is the number of letters in the string.

⌘ T w o Ways to determine string length.

☒ Tokenizing the string OR

☒ Counting individual symbols

Note: by which we should count the string length is indicated in the question

Length of Strings

⌘ Example1: (Tokenizing)

$\Sigma = \{a, b\}$

$s = \text{"ababa"}$

tokens = (a)(b)(a)(b)(a)

$|s| = 5$

Length of Strings

⌘ Example2: (Tokenizing)

$\Sigma = \{B, aB, bab, d\}$

$s = \text{"BaBbabBd"}$

Tokenizing = (B), (aB), (bab), (B), (d)

$|s| = 5$

Note: also the string contains 8 number of letters. See in next example3.

Length of Strings

⌘ Example 3: (Counting individual letters)

$\Sigma = \{B, aB, bab, d\}$

$s = \text{"BaBbabBd"}$

$|s| = 5$

Practice Exercise:

- Let $\Sigma = \{a, r, n, o, l, d\}$. List down 3 languages (L) of length 3 such that L does not contain a vowel.
- $L_1 = rnl$
- $L_2 = nld$
- $L_3 = dln$

Reverse of a String

⌘ Definition:

The reverse of a string s denoted by $\text{Rev}(s)$ or $\mathbf{s}^r$, is obtained by writing the letters of s in reverse order.

⌘ Example:

If $s=abc$ is a string defined over $\Sigma=\{a,b,c\}$
then $\text{Rev}(s)$ or $\mathbf{s}^r = \text{cba}$

⌘ Example:

Composite symbols are not reversed further

$$\Sigma = \{B, aB, bab, d\}$$

$$s = BaBbabBd$$

$$\text{Rev}(s) = d \ B \ bab \ aB \ B$$

Valid / Invalid Alphabets

⌘ While defining an alphabet of letters consisting of more than one symbols, no letter should be started with the letter of the same alphabet.

Valid / Invalid Alphabets

- ⌘ one letter should not be the prefix of another.
- ⌘ one letter may be the suffix of another.

Valid / Invalid Alphabets

⌘ Example 1

⌘ Consider an alphabet

$\Sigma_1 = \{B, aB, bab, d\}$ and a string Bababd.

☐ (B), (abab), (d)

☐ (B), a, (bab), (d)

Valid / Invalid alphabets

⌘ Example 2:

⌘ N o w consider another alphabet

$\Sigma_2 = \{B, Ba, bab, d\}$ and a string Bababd.

☐ (Ba), (bab), (d)

☐ (B), (abab), (d)

Conclusion

$$\text{⌘} \Sigma_1 = \{B, aB, bab, d\}$$

$$\text{⌘} \Sigma_2 = \{B, \textcolor{red}{B}a, bab, d\}$$

Σ_1 is a valid alphabet while Σ_2 is an invalid alphabet and should be avoided.

Defining Formal Languages

⌘ Simple Tools for Defining Formal languages:

☑ **Descriptive definition(DD)**

☑ **Recursive definition(RD)**

☑ **Regular Expressions (RE)**

Descriptive Definition

In this method the formal language is defined, describing the conditions imposed on its words.

Descriptive Definition

⌘ Example 1:

The language L of strings of odd length, defined over $\Sigma=\{a\}$, can be written as

$$L=\{a, aaa, aaaaaa, \dots\}$$

Descriptive Definition

⌘ Example2:

The language L of strings that does not start with "a", defined over $\Sigma=\{a,b,c\}$, can be written as:

$$L=\{b, c, ba, bb, bc, ca, cb, cc, \dots\}$$

Descriptive Definition

⌘ Example3:

The language L of all strings of length 2, defined over $\Sigma=\{0,1,2\}$, can be written as:

$$L=\{00, 01, 02, 10, 11, 12, 20, 21, 22\}$$

Descriptive Definition

⌘ Example4:

The language L of strings ending in 0, defined over $\Sigma=\{0,1\}$, can be written as:

$$L=\{0,00,10,000,010,100,110,\dots\}$$

Descriptive Definition

⌘ Example 5:

The language **EQUAL**, of strings with number of *a*'s equal to number of *b*'s, defined over $\Sigma=\{a,b\}$, can be written as:

$$\text{EQUAL} = \{\Lambda, ab, aabb, abab, baba, abba, bbbbaaa, \dots\}$$

Descriptive Definition

⌘ Example 6:

The language **EVEN-EVEN**, of strings with even number of a's and even number of b's, defined over $\Sigma=\{a,b\}$, can be written as

$\{\Lambda, aa, bb, aaaa, aabb, abab, abba, baab, baba, bbaa, bbbb, \dots\}$

Descriptive Definition

⌘ Example 7:

The language **INTEGER**, of strings defined over $\Sigma = \{-, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, can be written as

$$\text{INTEGER} = \{\dots, -2, -1, 0, 1, 2, \dots\}$$

Descriptive Definition

⌘ Example8:

The language **EVEN**, of strings defined over $\Sigma=\{-,0,1,2,3,4,5,6,7,8,9\}$, can be written as

$$\text{EVEN} = \{ \dots, -4, -2, 0, 2, 4, \dots \}$$

Descriptive Definition

⌘ Example9:

The language $\{a^n b^n\}$, of strings defined over $\Sigma=\{a,b\}$, as

$\{a^n b^n : n=1,2,3,\dots\}$, can be written as:

$\{ab, aabb, aaabbb, aaaabbbb, \dots\}$

Descriptive Definition

⌘ Example 10:

The language $\{a^n b^n a^n\}$, of strings defined over $\Sigma = \{a, b\}$, as

$\{a^n b^n a^n : n = 1, 2, 3, \dots\}$, can be written as:

$\{aba, aabbaa, aaabbbbaaa, aaaabbbbbaaaaa, \dots\}$

Descriptive Definition

⌘ Example-11:

The language **factorial**, of strings defined over $\Sigma = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

factorial = $\{1, 2, 6, 24, 120, \dots\}$

Descriptive Definition

⌘ Example 12:

The language **FACTORIAL**, of strings defined over $\Sigma=\{a\}$, as

$\{a^{n!} : n=1,2,3,\dots\}$, can be written as $\{a,aa,aaaaaa,\dots\}$.

It is to be noted that the language FACTORIAL can be defined over any single letter alphabet only.

Descriptive Definition

⌘ Example 13:

The language **DOUBLEFACTORIAL**, of strings defined over $\Sigma=\{a, b\}$, as

$\{a^n b^n : n=1,2,3,\dots\}$, can be written as

$\{ab, aabb, aaaaaabbbbbbb,\dots\}$

Descriptive Definition

Example14:

The language **SQUARE**, of strings defined over $\Sigma=\{a\}$, as

$\{a^{n^2} : n=1,2,3,\dots\}$, can be written as

$\{a, aaaa, aaaaaaaaaa, \dots\}$ (i.e. $1^2, 2^2, 3^2, 4^2, \dots$)

Descriptive Definition

Example15:

The language **DOUBLESQUARE**, of strings defined over $\Sigma=\{a,b\}$, as $\{a^{n^2} b^{n^2} : n=1,2,3,\dots\}$, can be written as

$\{ab, aaaabbbb, aaaaaaaaaabbbbbbbbbbb, \dots\}$

Descriptive Definition

⌘ Example 16:

The language **PRIME**, of strings defined over $\Sigma=\{a\}$, as

$\{a^p : p \text{ is prime}\}$, can be written as

Prime nos: 1, 2, 3, 5, 7, 11, 13,...

$\{a, aa, aaa, aaaaa, aaaaaaa, aaaaaaaaaaaaaa...\}$

Descriptive Definition



Example17:

PALINDROME:

Examples: madam, aba,...

The language consisting of Λ and the strings s defined over Σ such that $\text{Rev}(s)=s$.

Descriptive Definition

⌘ Example-17:
PALINDROME:

It is to be denoted that the words of PALINDROME are called palindromes.

For $\Sigma = \{a, b\}$,

$\text{PALINDROME} = \{\Lambda, a, b, aa, bb, aaa, aba, bab, bbb, \dots\}$

Note:

Generalized Rule on the number of strings

⌘ Number of strings of length 'm' defined over alphabet of 'n' letters is n^m .

Where:

n = string length and

m = number of alphabets

Note:

Generalized Rule on the number of strings

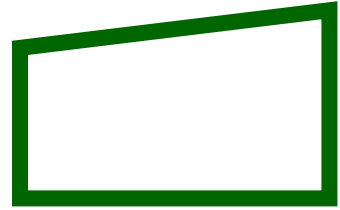
- Example 1:

-  The language of strings of length 2, defined over $\Sigma=\{a,b\}$ is;
- $L=\{aa, ab, ba, bb\}$
- *i.e.* number of strings = 2^2

Note:

Generalized Rule on the number of strings

- Example2:
 -  The language of strings of length 3, defined over $\Sigma=\{a,b\}$ is:
 - $L=\{aaa, aab, aba, baa, abb, bab, bba, bbb\}$
 - *i.e.* number of strings = 2^3



Module 1: Introduction to Automata Theory & Formal Languages

Professor: Dr. Arnold B. Galve
abgalve@kld.edu.ph